Программирование в лингвистике

Лекция 5

Структурное программирование. Циклы

Структурирование: понятия

1.
2.

Последовательность

Команды выполняются в том порядке, в котором они представлены в коде

if

Ветвление

Условные конструкции, создающие ветки

Цикл

Команды выполняются повторно либо заданное кол-во раз, либо до тех пор, пока условие равно True

Цикл for

Тело цикла:

```
for <переменная цикла> in <итерируемый объект>:
    команда
    команда
else:
    команда
```

Используется, когда:
- Мы знаем, сколько раз нам повториться
- Мы перебираем элементы итерируемого объекта

Циклу for нужен итерируемый объект – объект, который может возвращать элементы по одному.
+ Списки, строки, кортежи, множества, словари, итераторы, генераторы

Итерируемся по строке:

```
for letter in "abcd":
  print(letter)
```

```
a
b
c
d
```

range()

```
for i in range(3, 0, -1):
  print(i)
```

```
3
2
1
```

Range

```
print("0123"[3:0:-1])
```

```
321
```

Срезы строки

Команда range() создает итератор, который возвращает целые числа в заданном диапазоне с заданным шагом:

range(начало, конец, шаг)

Диапазон и шаг работают как срезы в строке:

тут 3 = начало, 0 = конец, -1 = шаг.

for i in range(n) часто используется при работе с последовательным вводом данных

for i in range(n) – пример использования

```
n = int(input("Введите число: "))
summa = 0
for i in range(n):
    x = int(input("Введите значение: "))
    summa += 1/x
harm_mean = n/summa
print(harm_mean)
```

Подсчет гармонического среднего простого из n значений:
- Цикл for выполняет n итераций по диапазону, заданному в range.
- В каждой итерации значение считывается и сразу обрабатывается.

* Гармоническое среднее простое используется только с положительными величинами

Вложенный цикл for

```
string = "ab"
for letter in string:
  for i in range(len(string)):
    print(f'{letter}{string[i]}')
```

```
aa
ab
ba
bb
```

Первый цикл итерируется напрямую по символам в строке ‘ab’.
Вложенный цикл итерируется по символам в сроке ‘ab’, но уже по их индексам (для разнообразия).
Переменные обоих циклов сочетаются в print, и на каждой итерации вложенного цикла печатаются различные комбинации символов.

Генерация списков

Генераторы списков (list comprehensions) в Python — это лаконичный способ создания списков с использованием цикла for.

```
n = int(input("Введите число: "))
lst = [x**2 for x in range(n) if x % 2 == 0]
```

Генераторы списков зачастую работают быстрее явных циклов for, использующих .append()

- Позволяет в одну строку кода создавать и заполнять списки.
- Обязательный синтаксис включает выражение для генерации элемента и итерацию по заранее заданному объекту.
- Может также включать фильтр по условию после if.

Цикл while

Тело цикла:

```
while <условие выполнения>:
    команда
    команда
else:
    команда
```

Циклу while нужно условие, которое будет заставлять его повторять команды. Если условие изначально False, срабатывает else. Если условие становится False в процессе работы цикла, то цикл останавливается по завершении итерации и приступает к телу else.

Используется, когда:

- Мы не знаем, сколько раз нам повториться
- Есть условие для выполнения цикла

Пример такого цикла:

```
number = input()
while not number.isdigit():
  number = input('Введите число!\n')
else:
  print('Наконец-то число!')
```

Цикл for, вложенный в while

```
string = input("Введите строку: ")
while string:
  for letter in string:
    print(letter)
  string = input("Введите строку: ")
else:
  print("Цикл закончен")
```

```
Введите строку: эй
э
й
Введите строку:
Цикл закончен
```

Цикл while запрашивает строку до тех пор, пока пользователь не нажимает на Enter (это будет пустая строка = False).

Цикл for печатает эту строку побуквенно.

Когда пользователь ввел пустую строку, программа печатает «Цикл закончен».

Отступы в Python

```
string = input("Введите строку: ")
while string:
  if len(string) <= 2:
    for letter in string:
      print(letter)
    else:
      print(f"Мы напечатали букв: {len(string)}")
  else:
    print("Слишком много букв")
  string = input("Введите строку: ")
else:
  print("Цикл закончен")
```

Красное – цикл while

Синее – проверка

Зеленое – цикл for

While True

Исполнение цикла while требует bool, возвращающего True. Таким образом, если мы зададим неизменяемое условие наподобие while True, то из цикла мы сможем выйти только с помощью команды break или вручную прекратив исполнение кода. while False, как можно догадаться, никогда не выполнится.

ОПАСНО!

```
spisok = []
while True:
  spisok.append("a")
```

Подобный код будет бесполезно и беспощадно грузить Python-ядро, а также бесконечно потреблять оперативную память (с каждой итерацией все больше). Может вызвать зависание компьютера или даже вылет из системы.

Единственный способ прекратить его выполнение после запуска – прервать исполнение кода вручную.

break и continue

```
while True:
  number = input("Введите число: ")
  if number.isdigit():
    print("Молодчина!")
    break
  print('Я сказал число!')
```

break

Команда break прерывает работу цикла. Команды, следующие за ней в теле цикла, не выполняются. Следующая итерация не наступает.
Так, здесь будет бесконечно запрашиваться ввод до тех пор, пока пользователь не введет число.

Команда continue прерывает текущую итерацию и переходит к следующей. То, что следует за continue, не выполнится, но цикл не остановится.
Так, здесь, если пользователь ввел пробельный символ, цикл не дойдет до последнего print'а.

```
while True:
  number = input()
  if number.isdigit():
    break
  if number.isspace():
    print('нам не нужен пробел!')
    continue
  print('Вы ввели букву!')
```

continue

Обе команды применимы как для цикла while, так и для цикла for.

Спасибо за внимание!